Linear sorting algorithms

Linear sorting algorithms are algorithms that sort data in linear time, i.e., O(n). These algorithms are typically not comparison-based and are well-suited for specific types of data. Some well-known linear sorting algorithms include **Counting Sort**, **Radix Sort**, and **Bucket Sort**.

1. Counting Sort

Counting Sort works by counting the occurrences of each unique value in the input, and then using this information to place each value in the correct position in the output array. It's efficient when the range of input data is not significantly larger than the number of elements.

Example:

Let's sort the array A = [4, 2, 2, 8, 3, 3, 1].

Step-by-Step Process:

1. Find the maximum element in the array, which is 8.
2. Create a count array of size max + 1 (i.e., size 9) initialized with zeros: count = [0, 0, 0, 0, 0, 0, 0, 0, 0].
3. Traverse the input array and update the count array by incrementing the corresponding index:
    - For element 4, increment count[4].
    - For element 2, increment count[2] twice (since it appears twice).
    - For element 8, increment count[8], and so on. Final count array: count = [0, 1, 2, 2, 1, 0, 0, 0, 1].
4. Modify the count array such that each element at index i stores the sum of previous counts. This gives the positions in the sorted array: count = [0, 1, 3, 5, 6, 6, 6, 6, 7].
5. Traverse the input array in reverse order, and place elements in the correct position in the output array:
    - For element 1, place it at index 0 (count[1] - 1), and decrement count[1].
    - For element 3, place it at index 4 (count[3] - 1), and decrement count[3], and so on. Final sorted array:
      A = [1, 2, 2, 3, 3, 4, 8].

Time Complexity: O(n + k), where n is the number of elements and k is the range of input values.

---

2. Radix Sort

Radix Sort processes each digit of the numbers (starting from the least significant digit to the most significant). It uses a stable sorting algorithm, like Counting Sort, at each digit level.

Example:

Let's sort the array A = [170, 45, 75, 90, 802, 24, 2, 66].

Step-by-Step Process:

1. Start by sorting based on the least significant digit (units place):
    - Pass 1: Sorting by units place (using Counting Sort): A = [170, 90, 802, 2, 24, 45, 75, 66].
2. Move to the next digit (tens place):
    - Pass 2: Sorting by tens place: A = [802, 2, 24, 45, 66, 170, 75, 90].
3. Sort by the hundreds place:
    - Pass 3: Sorting by hundreds place: A = [2, 24, 45, 66, 75, 90, 170, 802].

Time Complexity: O(d * (n + k)), where d is the number of digits, n is the number of elements, and k is the range of digits (in base 10, k is 10).

---

3. Bucket Sort

Bucket Sort divides the elements into several buckets, sorts each bucket individually (usually with a comparison-based algorithm like Insertion Sort), and then merges the buckets to get the final sorted array.

Example:

Let's sort the array A = [0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51].

Step-by-Step Process:

1. Divide the elements into n buckets (here we assume 10 buckets for simplicity).
2. Distribute the elements into these buckets based on their value:
    - Element 0.42 goes into bucket 4 (because 0.42 * 10 = 4.2).
    - Element 0.32 goes into bucket 3 (because 0.32 * 10 = 3.2), and so on.
    - The buckets after distribution:
      Bucket 2: [0.23, 0.25],
      Bucket 3: [0.32],

Bucket 4: [0.42, 0.47],
Bucket 5: [0.52, 0.51].

3. Sort each bucket individually:
    - Bucket 2: [0.23, 0.25] (sorted).
    - Bucket 3: [0.32] (no sorting needed), and so on.
4. Concatenate all the sorted buckets to get the final sorted array: A = [0.23, 0.25, 0.32, 0.42, 0.47, 0.51, 0.52].

Time Complexity: O(n + k), where n is the number of elements and k is the number of buckets.

Summary

- **Counting Sort**: Efficient for small ranges of integers.
- **Radix Sort**: Suitable for sorting large numbers by digits.
- **Bucket Sort**: Ideal for uniformly distributed floating-point numbers.

These algorithms exploit specific properties of data and avoid comparison-based methods, leading to linear-time performance in the best-case scenarios.